

Title of Topic: Data Models – Introduction, Benefits, and Phases, ER
Diagrams – Symbols, Components, Relationships, Weak entities,
Attributes, Cardinality

Course Code: 23IT201

Course Title: Database Management Systems

Academic Year: 2024 - 2025 (Even Sem)

Module 1 – INTRODUCTION

1. Introduction to DBMS, Characteristics of DBMS, DBMS vs File Systems, need for DBMS.
2. Three Level DBMS Architecture.
3. **Data Models, Introduction, Benefits, and Phases. ER Diagrams – Symbols, Components, Relationships, Weak entities, Attributes, Cardinality.**
4. Relational Algebra, Domain Relational Calculus, Tuple Relational Calculus,
5. Keys - primary Key, Foreign Key.

Data Models – Introduction, Benefits, and Phases, ER Diagrams – Symbols, Components, Relationships, Weak entities, Attributes, Cardinality

Topics to be Covered

- Data Models
- Types of Data Models
- Physical Data Models
- Object Based Data Models
- Entity Relationship (ER) Data Model
- Object Oriented (OO) Data Model
- Record Based Data Models
- Hierarchical Data Model
- Network Data Model
- Relational Data Model
- ER Model to Relational Model Mapping

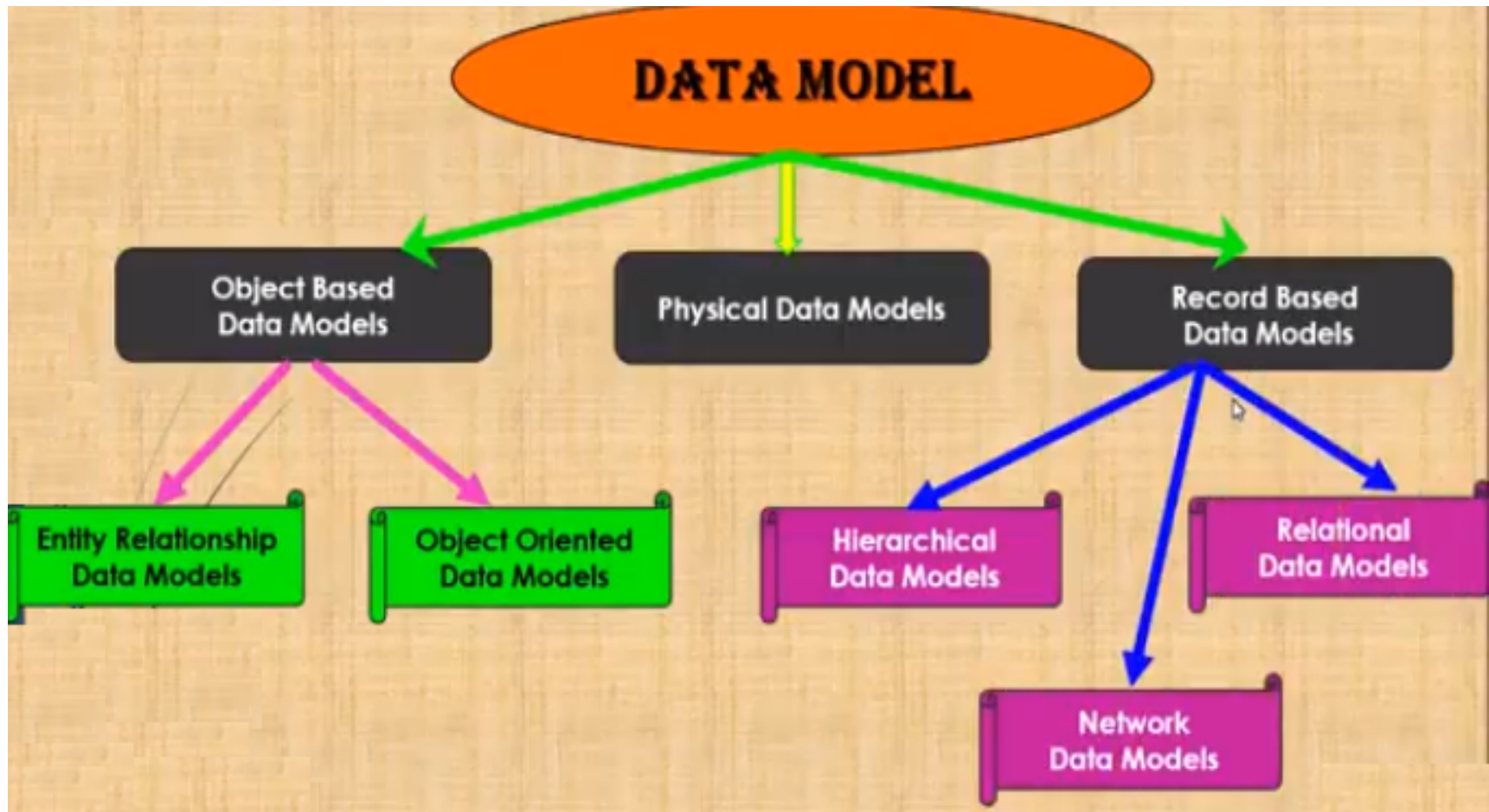
Data Models

- Define how the logical structure of a database is modelled
- Define how data is connected to each other
- Define how they are processed and stored in system
- Planning the structure of database is called data models
- Helps to put the real world requirements into a design
- Collection of conceptual tools for describing data, data relationships, data semantics and consistency constraints
- Helps to highlight any drawbacks of the plan and correct it at the design stage itself

Data Models:



Types of Data Models



Tools to draw Data Models (ER Diagram):



Physical Data Models

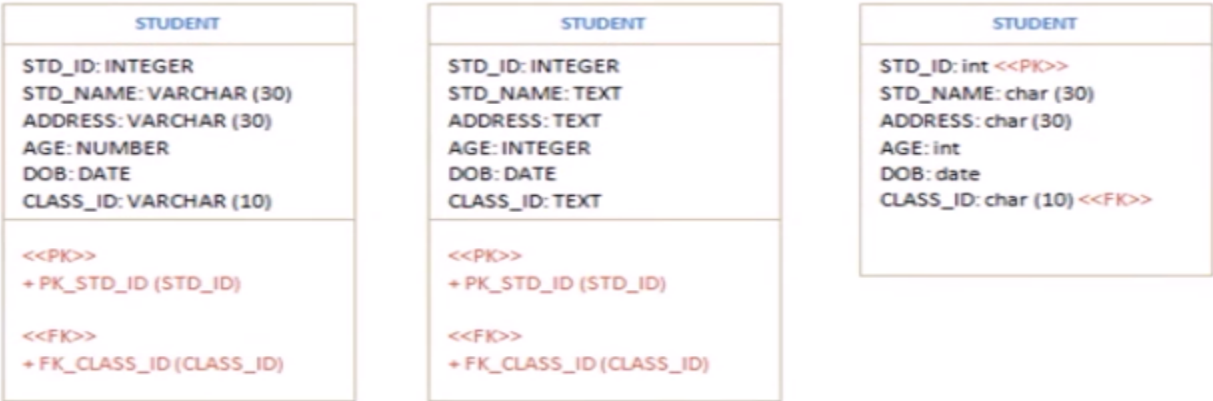
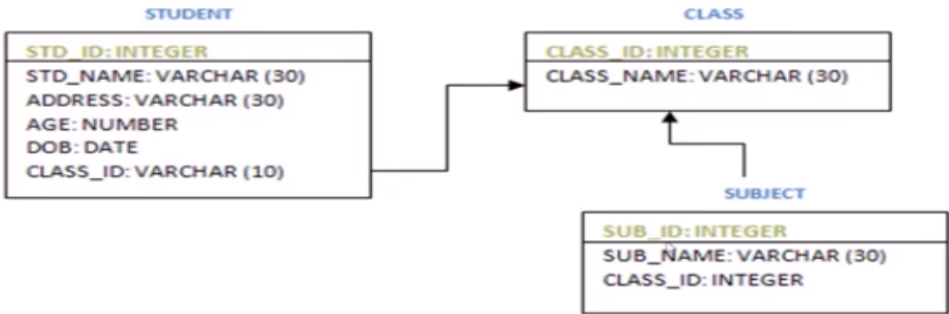
- Describe how data are stored, scattered and ordered in computer memory
- Describe how data would be retrieved from memory
- Represents the data at data layer or internal layer
- Represents each table, their columns and specifications, constraints like primary key, foreign key, etc...
- Basically represents how each tables are built and related to each other in DB
- Has tables and its specifications - Table names and their columns
- Has foreign keys - Used to represent the relationship between the tables
- Have de-normalized structure based on the user requirement
- Dependent on the RDBMS

Physical Data Models:



Physical Data Models

- Representation of physical data model diagram may be different, though it contains same information
- Depends on the user/designer how the diagram and the RDBMS servers are specified



Object Based Data Models

- Designed using the entities in the real world, attributes of each entity and their relationship
- Picks up each thing/object in the real world which is involved in the requirement
- Types
 - ✓ Entity Relationship Data Model
 - ✓ Object Oriented Data Model

Object Based Data Models:



Entity Relationship (ER) Data Model

- Represents the entities, attributes and their relationship in the form of picture to make the developer understand the system better
- Graphical representation of real world objects with their attributes and relationship
- Makes the system easily understandable
- Considered as a top down approach of designing a requirement
- Developed to facilitate database design by allowing specification of an enterprise schema that represents the overall logical structure of a database

Entity Relationship (ER) Data Model

- Very useful in mapping the meanings and interactions of real-world enterprises onto a conceptual schema and because of this many database-design tools draw on concepts from the ER model
- Employs three basic concepts such as entity sets, relationship sets, and attributes
- Has an associated diagrammatic representation, the ER diagram, which can express the overall logical structure of a database graphically

Entity Sets

- An object that exists and is distinguishable from other objects
- Entity is a real or abstract object (depends on requirement)
- Maybe Person, Place or Building – identifier
- Part of an enterprise that is of interest. E.g., Person, Company, Event, Plant

Entity Relationship (ER) Data Model

Entity Sets

- Set of entities of the same type that share the same properties. E.g., Set of all Persons, Companies, Trees, Holidays
 - ✓ Student entity set might represent the set of all students in the university
- An entity is represented by a set of attributes; i.e., descriptive properties possessed by all members of an entity set
- E.g., instructor = (ID, name, street, city, salary)
course = (course_id, title, credits)
- A subset of the attributes form a primary key of identifying each member of the set

<u>instructor_ID</u>	<u>instructor_name</u>	<u>student-ID</u>	<u>student_name</u>
76766	Crick	98988	Tanaka
45565	Katz	12345	Shankar
10101	Srinivasan	00128	Zhang
98345	Kim	76543	Brown
76543	Singh	76653	Aoi
22222	Einstein	23121	Chavez
		44553	Peltier

instructor

student

Entity Relationship (ER) Data Model

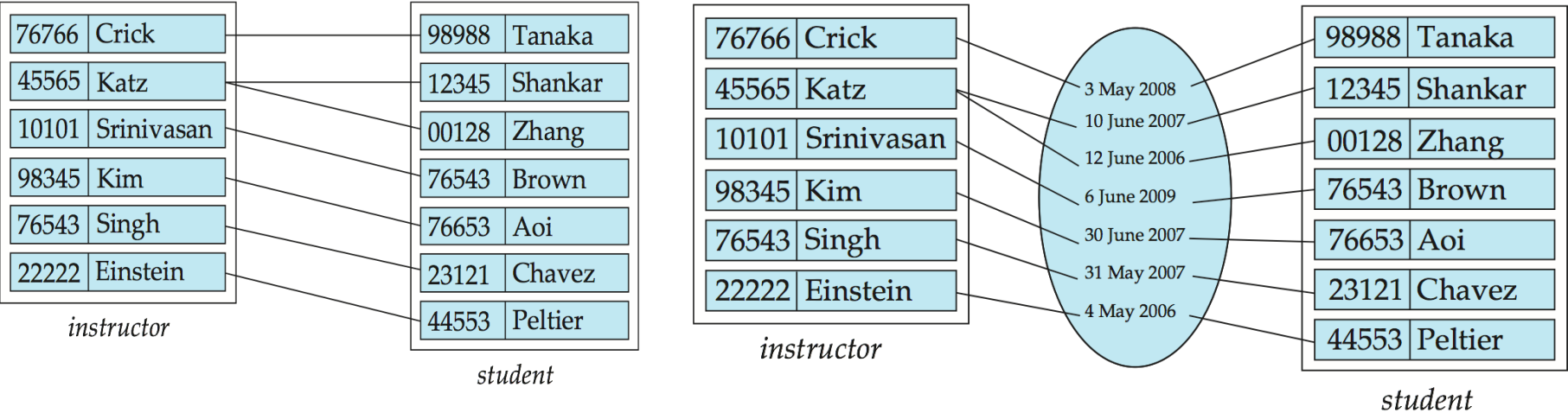
Relationship Sets

- A relationship is an association among several entities
- E.g., A relationship advisor can be defined that associates instructor Katz with student Shankar. This relationship specifies that Katz is an advisor to student Shankar
- A relationship set is a set of relationships of the same type. Formally, it is a mathematical relation on $n \geq 2$ (possibly non distinct) entity sets. If $E_1, E_2, \dots, E_n$ are entity sets, then a relationship set R is a subset of $\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$; where $(e_1, e_2, \dots, e_n)$ is a relationship
- An attribute can also be associated with a relationship set

Entity Relationship (ER) Data Model

Relationship Sets

- For instance, the advisor relationship set between entity sets instructor and student may have the attribute date which tracks when the student started being associated with the advisor



Entity Relationship (ER) Data Model

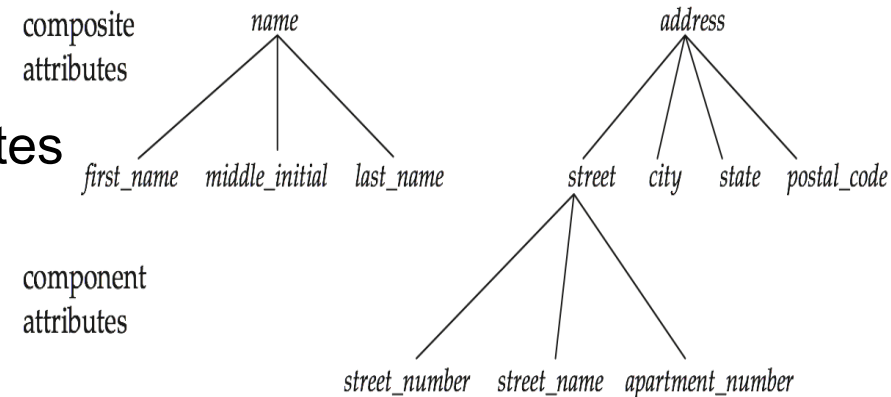
Relationship Types

- Relationships are associations or connections that exist between entities and may be looked at as mappings between two or more entity sets
- Although we allow relationships among any number of entity sets, the most common cases are binary relationships between two entity sets
- Mapping two or more entity sets. E.g., Player and Match
- Three types of relationship
 - ✓ Binary Relationship
 - ✓ Recursive Relationship
 - ✓ Ternary Relationship

Entity Relationship (ER) Data Model

Attributes

- For each attribute, there is a set of permitted values, called the domain, or value set, of that attribute. E.g., Domain of attribute semester might be strings from the set {Fall, Winter, Spring, Summer}
- Attribute types:
 - Simple or composite attributes.
 - Single-valued or multivalued attributes
 - Stored or Derived
 - Required or Optional
- Simple: Attributes not sub divided. E.g., Country, Place of Birth
- Composite: Attributes that are sub divided. E.g., Dob, Name, Address
- Single Valued: Many attributes have a single value. E.g., Regno



Entity Relationship (ER) Data Model

Attribute types:

- Simple or composite attributes.
- Single-valued or multivalued attributes
- Example: multivalued attribute: phone_numbers
- Derived attributes
- Can be computed from other attributes
- Example: age, given date_of_birth
- - Stored or Derived
 - Required or Optional

Entity Relationship (ER) Data Model

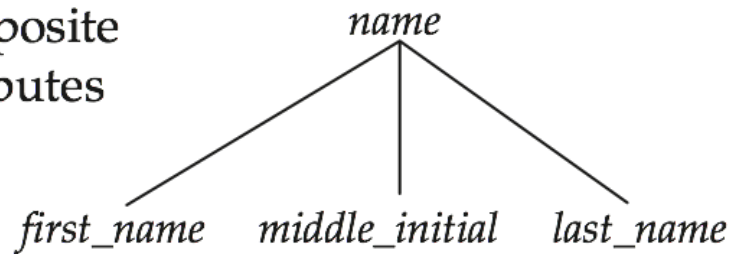
Attributes

Simple or composite

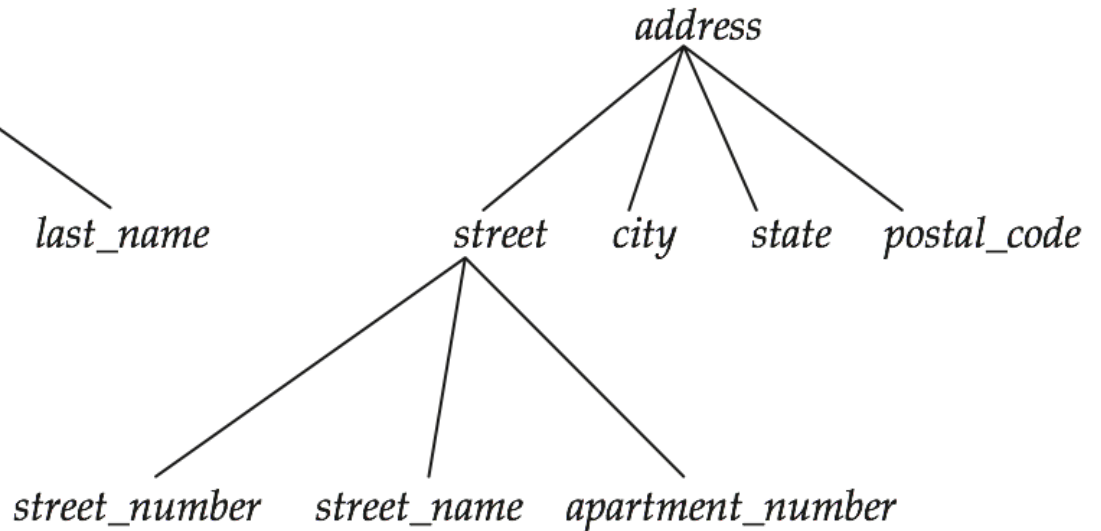
- **Multi-valued:** Some attributes have multiple values. E.g., Umpire - TV, Referee, Reserve Umpire
- **Single Valued :** regno
- **Stored :** Attribute values that are related. E.g., Players Career, Batting Average
- **Derived:** Value for this type of attribute can be derived from the values of other related attributes or entities. E.g., Age derived from DOB
- **Required:** Necessary. E.g., match ID
- **Optional:** It may or may not be necessary. E.g., An attribute takes a null value when an entity does not have a value for it

Composite Attributes

composite
attributes



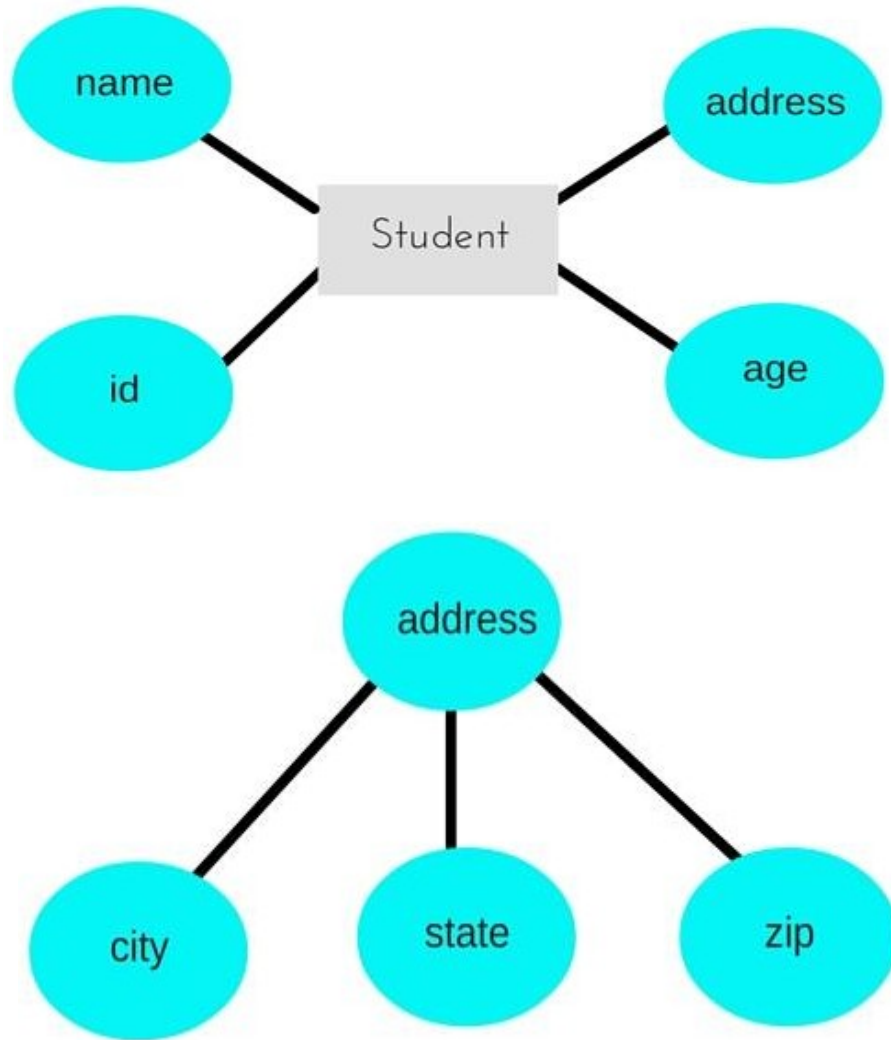
component
attributes



Simple : attributes not sub divided Eg: country, place of birth

Composite : attributes that are sub divided Eg: dob, Name

Composite Attributes



Entity Relationship (ER) Data Model

Mapping Cardinality Constraints

- Mapping Cardinality or Cardinality Ratios express the number of entities to which another entity can be associated via a relationship set
- Maximum number of relationship instances that an entity can participate in.
E.g., Employee and Office
- Most useful in describing binary relationship sets
- For a binary relationship set the mapping cardinality must be one of the following types:

▣ one to one: 

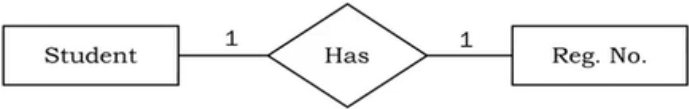
▣ one to many: 

▣ many to many: 

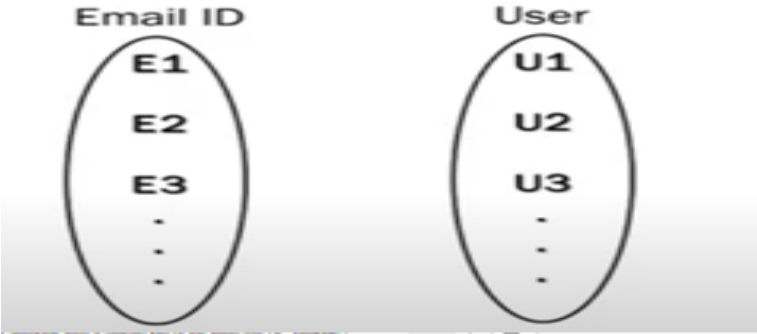
Entity Relationship (ER) Data Model

Mapping Cardinality Constraints

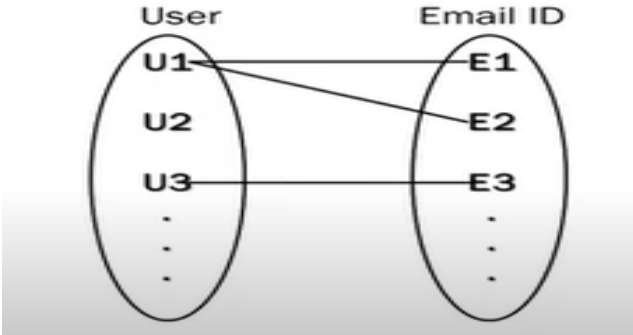
✓ One to One



✓ Many to One



✓ One to Many

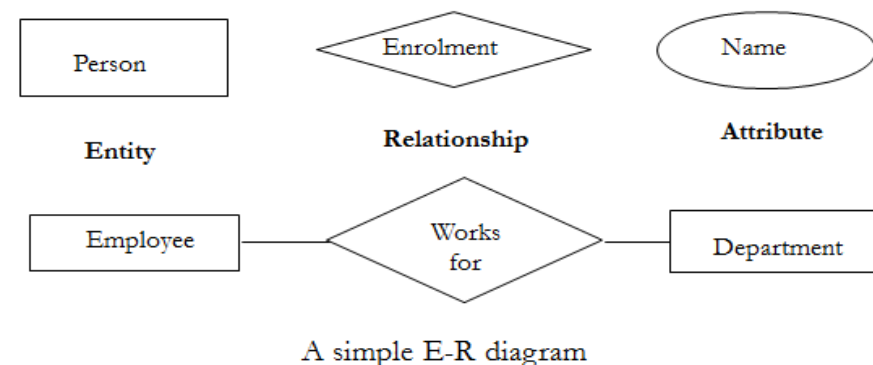
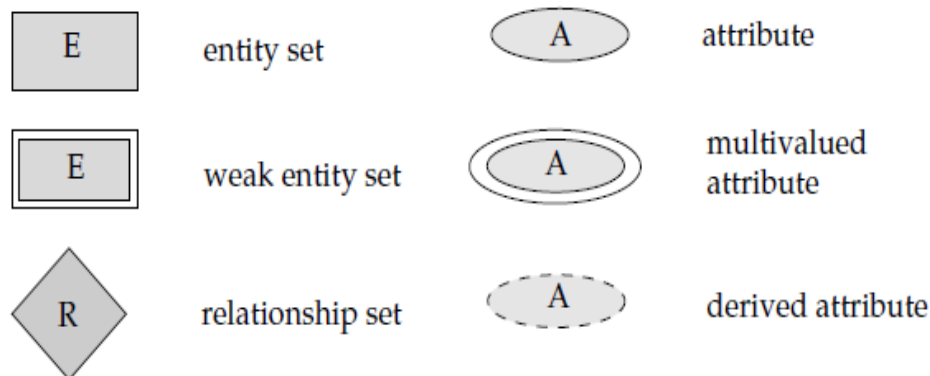


✓ Many to Many

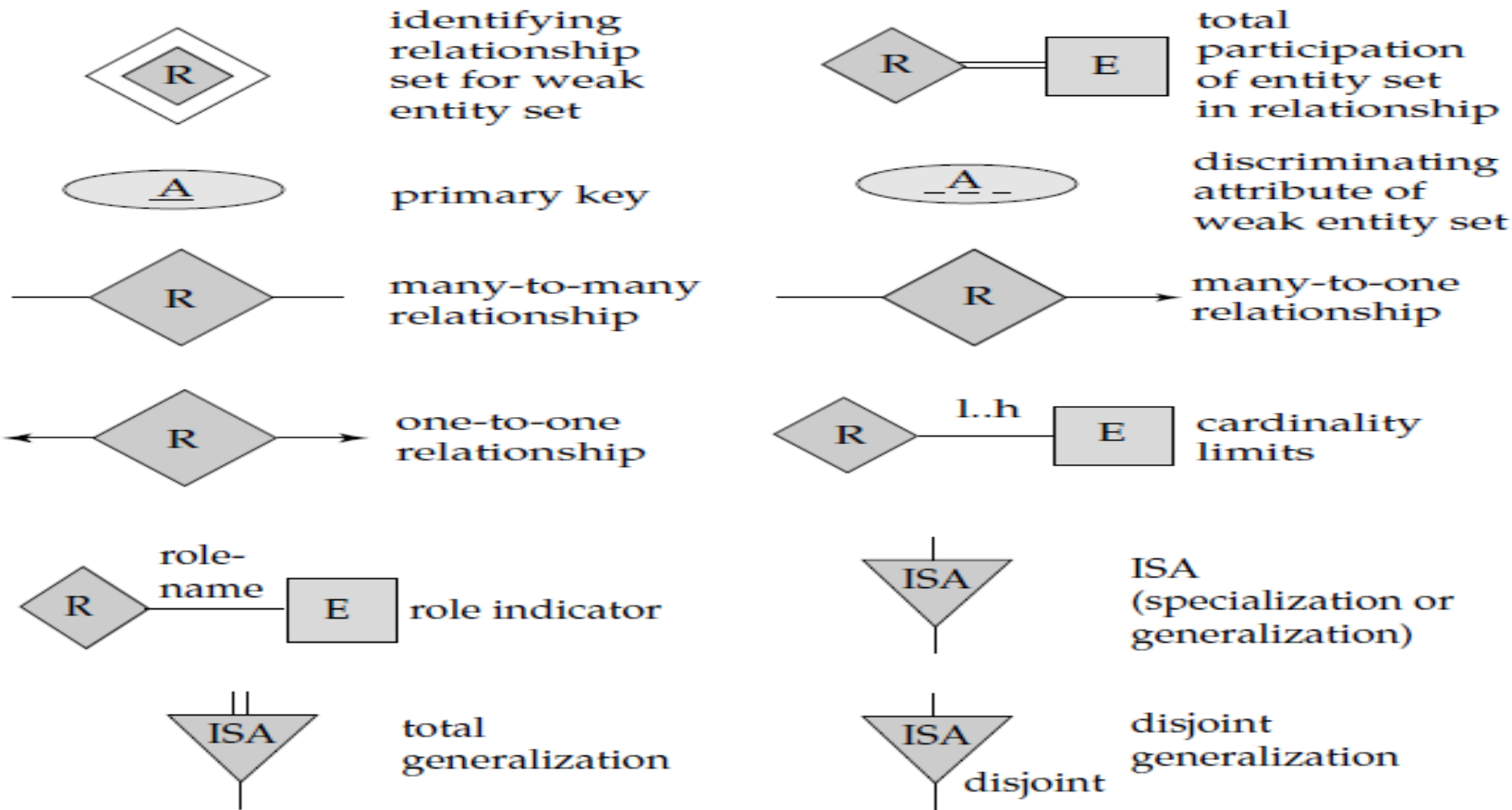


Entity Relationship (ER) Data Model

1. Start by identifying those entity sets that should be included
2. Choose the appropriate attributes for entity sets. E.g., instructor entity set, we will include the attributes ID, name, dept name, and salary
3. The relationship sets among the various entities are formed
4. These relationship sets may result in a situation where attributes in the various entity sets are redundant and need to be removed from the original entity sets



Entity Relationship (ER) Data Model

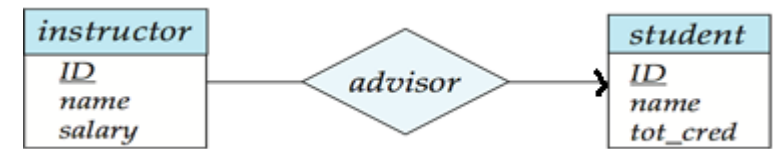


Entity Relationship (ER) Data Model

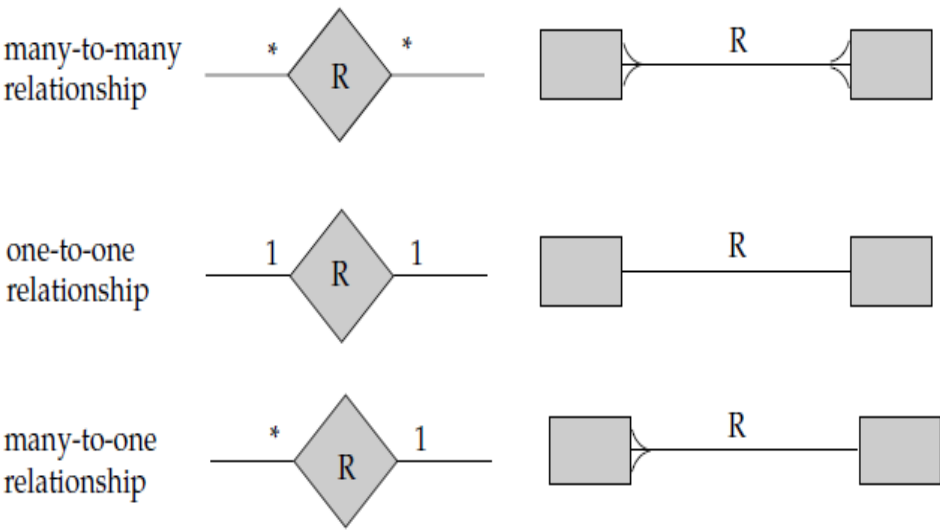
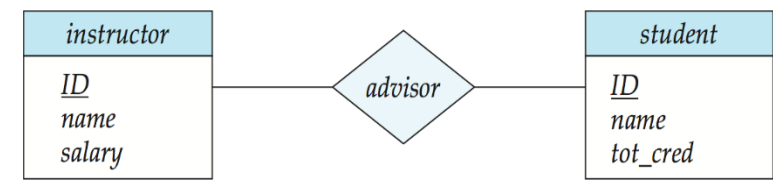
One-to-Many Relationship



Many-to-One Relationship



Many-to-Many Relationship



Entity Relationship (ER) Data Model

Total Participation (indicated by double line)

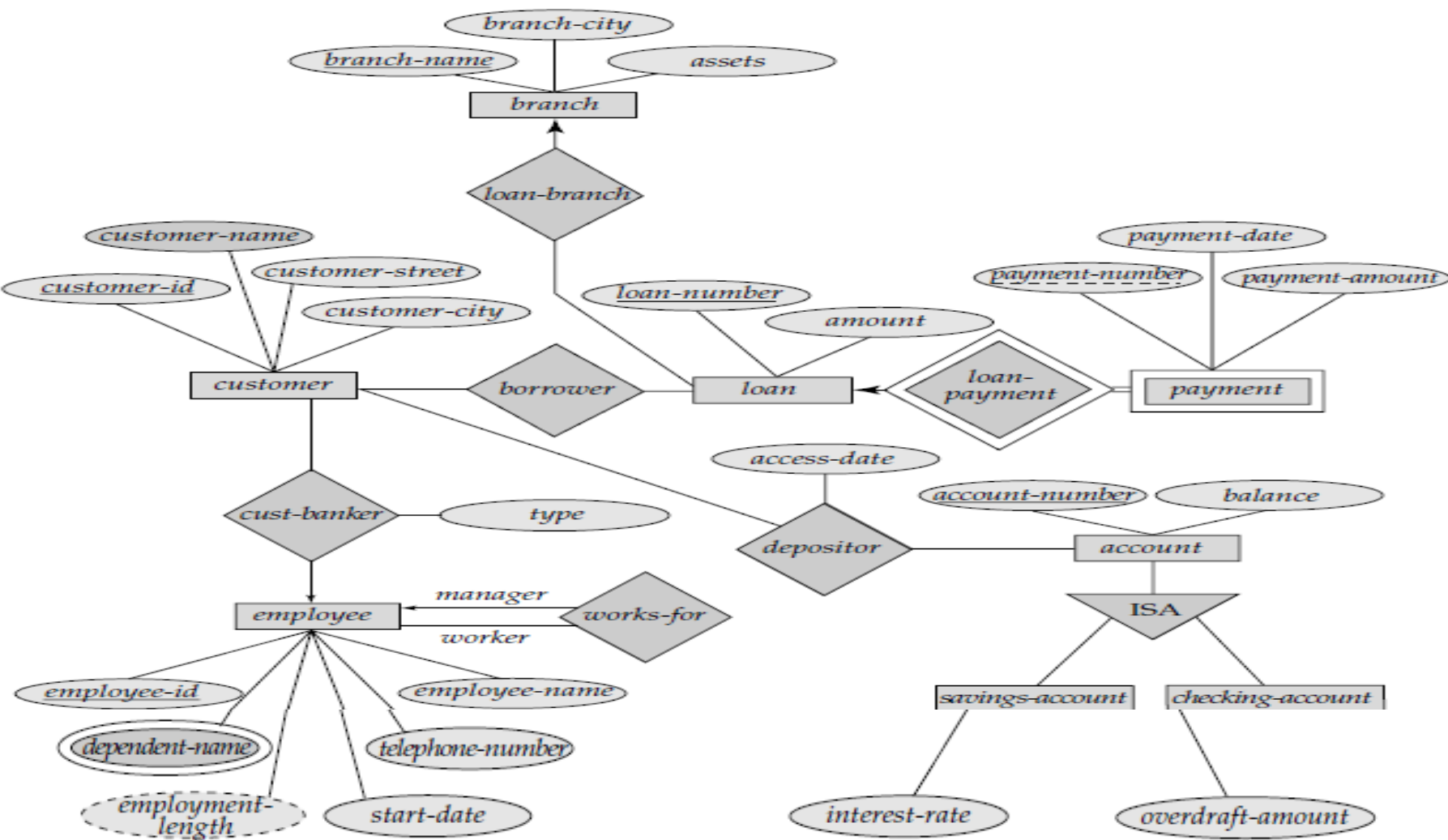
- Every entity in the entity set participates in at least one relationship in the relationship set.
- E.g., Participation of student in advisor relation is total - Every student must have an associated instructor

Partial Participation (indicated by single line)

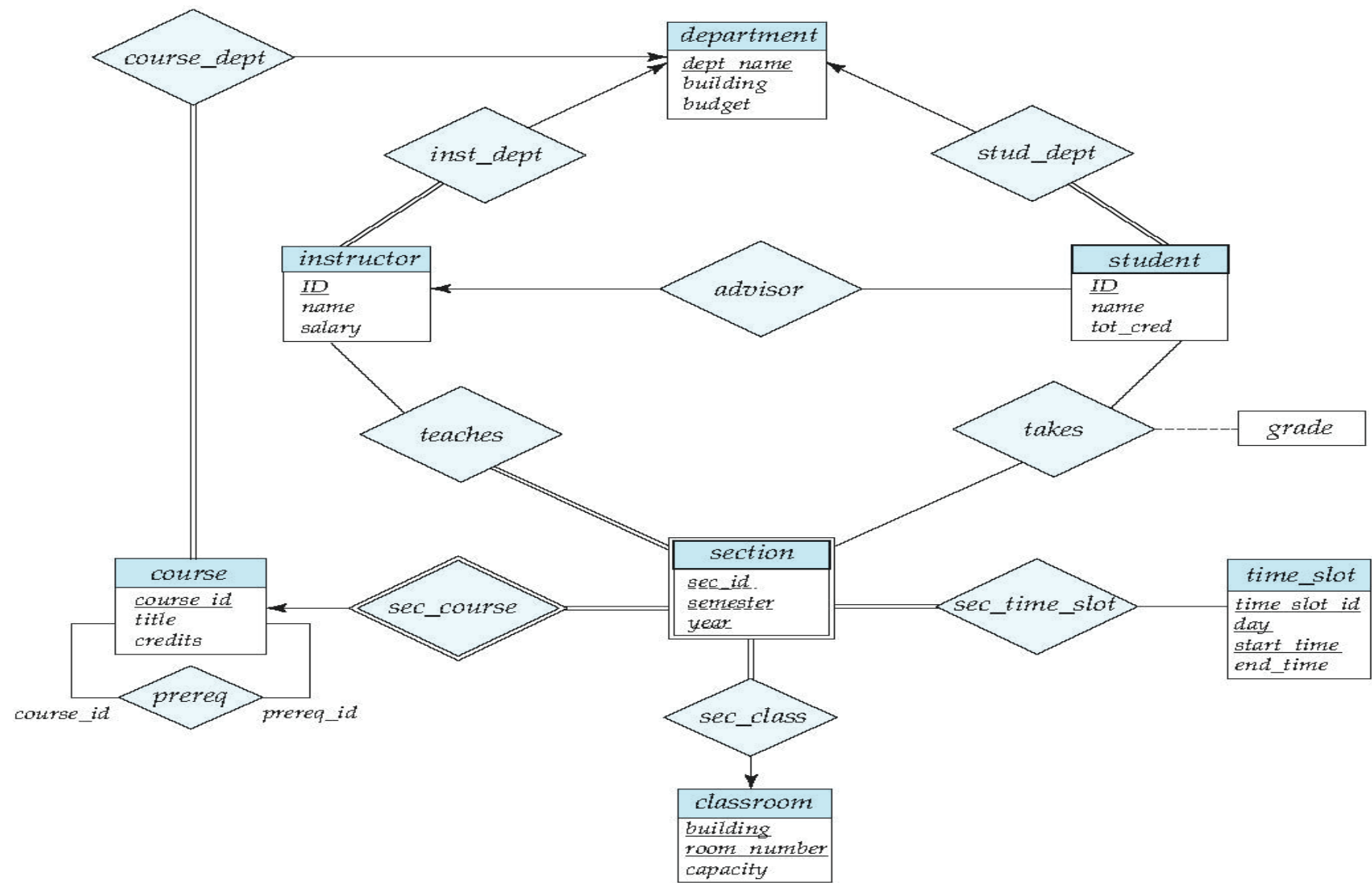
- Some entities may not participate in any relationship in the relationship set
- E.g., Participation of instructor in advisor is partial



Entity Relationship (ER) Data Model



Entity Relationship (ER) Data Model



Entity Relationship (ER) Data Model

Redundant Attributes

- Each instructor has an associated department using a relationship set inst_dept and attribute dept_name appears in both entity sets as shown:
 - ✓ instructor, with attributes: ID, name, dept_name, salary
 - ✓ department, with attributes: dept_name, building, budget
- As it is the primary key for the entity set department, it replicates information present in the relationship and is therefore redundant in the entity set instructor and needs to be removed
- When converting back to tables, in some cases the attribute gets reintroduced

Entity Relationship (ER) Data Model

Weak Entity Sets

- Entity set that does not have sufficient attributes to form a primary key
- Entity set that has a primary key is termed a strong entity set
- Consider a section entity, which is uniquely identified by a **course_id, semester, year, and sec_id**
- Clearly, section entities are related to course entities. Suppose a relationship set **sec_course** is created between entity sets section and course
- Here, **sec_course** is redundant, since section already has an attribute **course_id**, which identifies the course with which the section is related
- One option to deal with this redundancy is to get rid of the relationship **sec_course**; however, by doing so the relationship between section and course becomes implicit in an attribute, which is not desirable

Entity Relationship (ER) Data Model

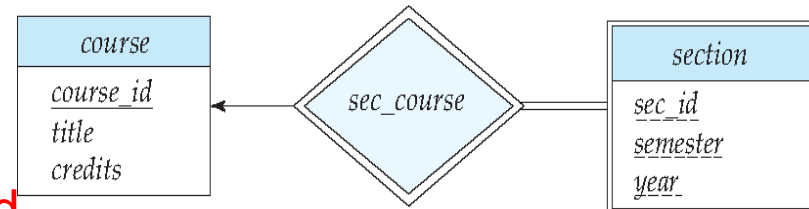
Weak Entity Sets

- An alternative way to deal with this redundancy is to not store the attribute `course_id` in the section entity and to only store the remaining attributes `section_id`, `year`, and `semester`
- The entity set section does not have enough attributes to identify a particular section entity uniquely; although each section entity is distinct, sections for different courses may share the same `section_id`, `year`, and `semester`
- To deal with this problem, the relationship `sec_course` is treated as a special relationship that provides extra information, in this case, the `course_id`, required to identify section entities uniquely
- The notion of weak entity set formalizes the above intuition and a weak entity set is depicted via a double rectangle

Entity Relationship (ER) Data Model

Weak Entity Sets

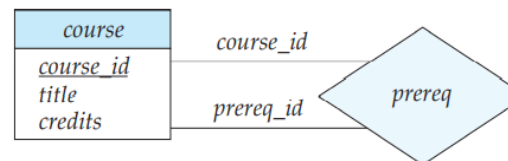
- The discriminator of a weak entity set is shown with a dashed line
- The relationship set connecting the weak entity set to the identifying strong entity set is depicted by a double diamond



- Key will be - **course_id, semester, year, and sec_id**

Unary Relationship

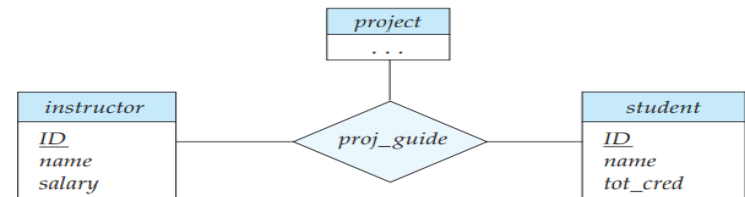
- The role indicators `course_id` and `prereq_id` between the course entity set and the prereq relationship set are indicated as shown



Entity Relationship (ER) Data Model

Nonbinary Relationship Sets (Ternary relationship)

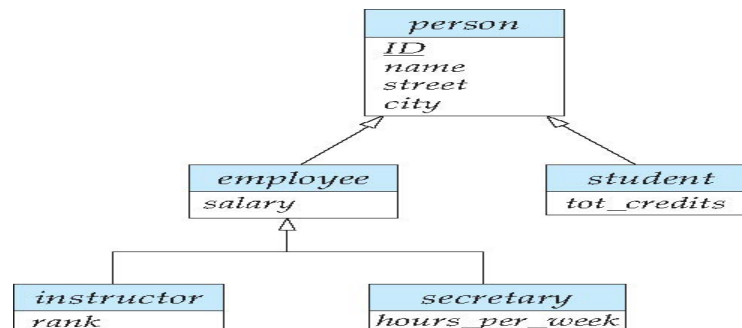
- Consists of the three entity sets instructor, student, and project, related through the relationship set proj_guide



Extended E-R Features: Specialization

- An entity set may include subgroupings of entities that are distinct in some way from other entities in the set
- The process of designating subgroupings within an entity set is called specialization. E.g., person may be further classified as one of the following:

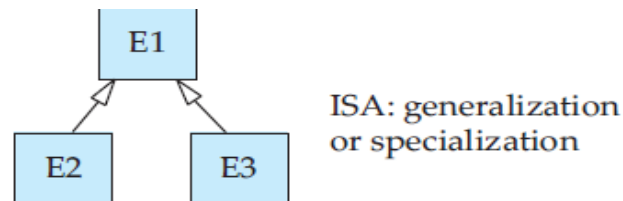
- ✓ employee
- ✓ student



Entity Relationship (ER) Data Model

Extended E-R Features: Generalization

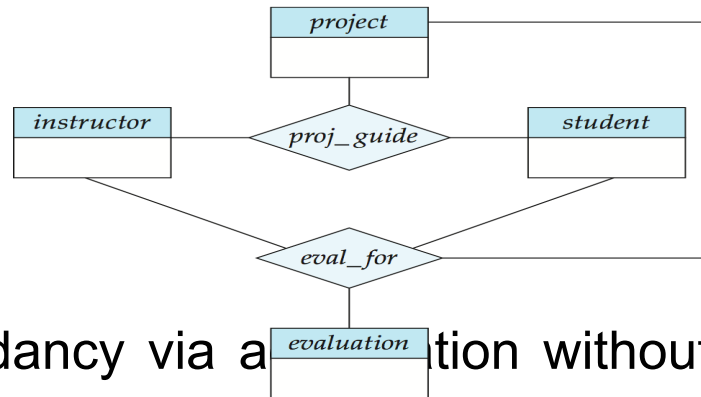
- A bottom-up design process – combine a number of entity sets that share the same features into a higher-level entity set
- The commonality can be expressed by generalization, which is a containment relationship that exists between a higher-level entity set and one or more lower-level entity sets
- Specialization and generalization are simple inversions of each other; they are represented in an E-R diagram in the same way and they are used interchangeably



Entity Relationship (ER) Data Model

Aggregation

- An abstraction through which relationships are treated as higher-level entities
- To record evaluations of a student by a guide on a project

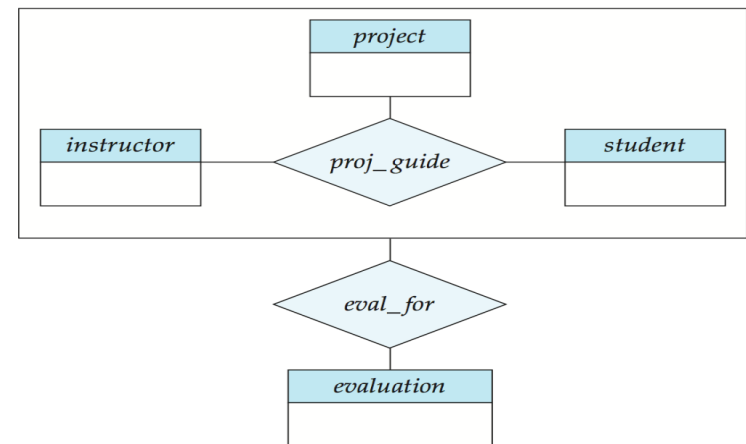


- Eliminate this redundancy via a *evaluation* entity without introducing redundancy, the following diagram represents
- A student is guided by a particular instructor on a particular project
- A student, instructor, project combination may have an associated evaluation

Entity Relationship (ER) Data Model

Aggregation

- Relationship sets `eval_for` and `proj_guide` represent overlapping information
- Every `eval_for` relationship corresponds to a `proj_guide` relationship
- However, some `proj_guide` relationships may not correspond to any `eval_for` relationships
- So it is not possible to discard the `proj_guide` relationship
- This redundancy can be eliminated via aggregation
- Treat relationship as an abstract entity
- Allows relationships between relationships
- Abstraction of relationship into new entity



Entity Relationship (ER) Data Model

Advantages

- Makes the requirement simple and easily understandable by representing simple diagrams
- One can convert ER diagrams into record based data model easily
- Easy to understand ER diagrams

Disadvantages

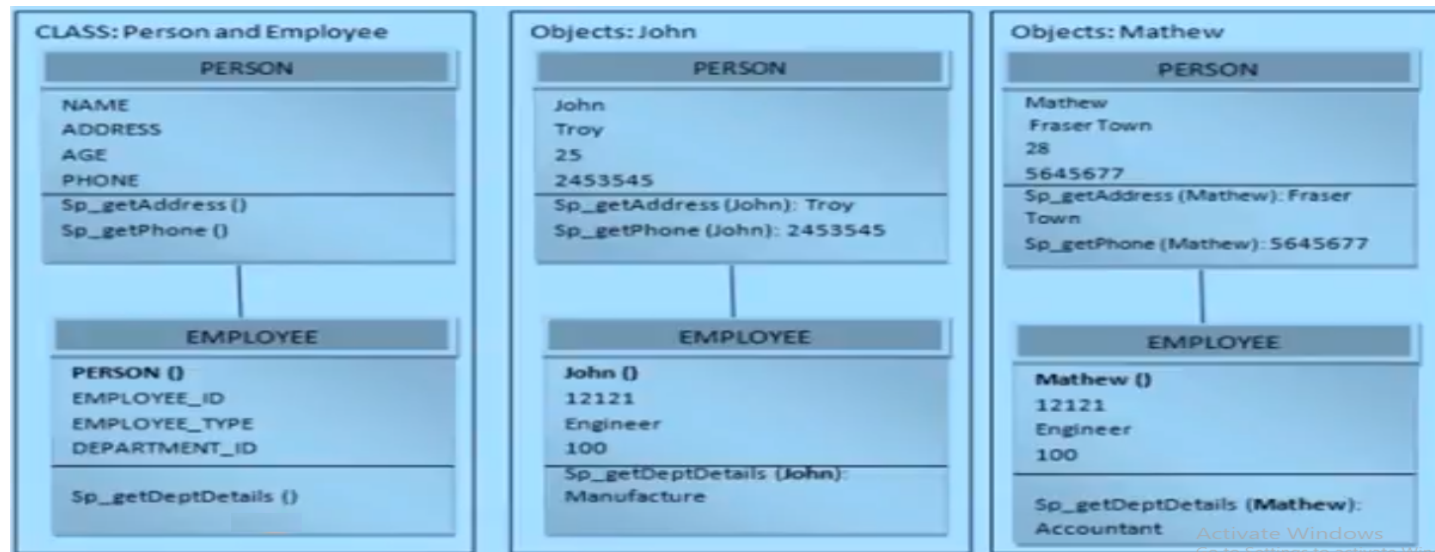
- No standard notations are available for ER diagram
- Flexibility in the notation depends upon the designer, how he draws it
- Meant for high level designs which cannot be simplified for low level design like coding

Object Oriented (OO) Data Model

- Considers each object in the world as objects and isolates it from each other
- Groups its related functionalities together and allows inheriting its functionality to other related sub-groups

Encapsulation

- Binds the features in one class and hides it from other classes
- Only visible to its objects and any inherited classes



Object Oriented (OO) Data Model

Advantages

- Inheritance property – Attributes and functionalities can be re-used
- Reduces the cost of maintaining the same data multiple times
- More flexible in the case of any changes
- Codes are re-used and OO model is more understandable

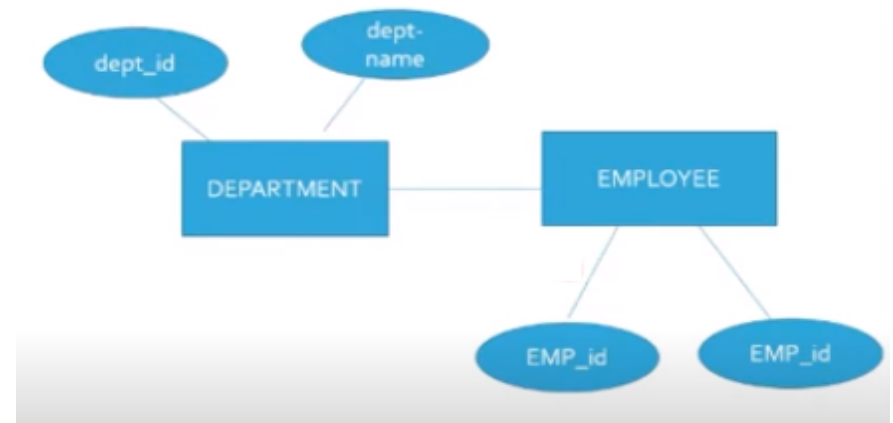
Disadvantages

- Not widely developed and complete to use it in the database systems.
Hence not accepted by the users
- Not a technology but a approach for solving the requirement. Hence, fails to put it in the DBMS

Record Based Data Models

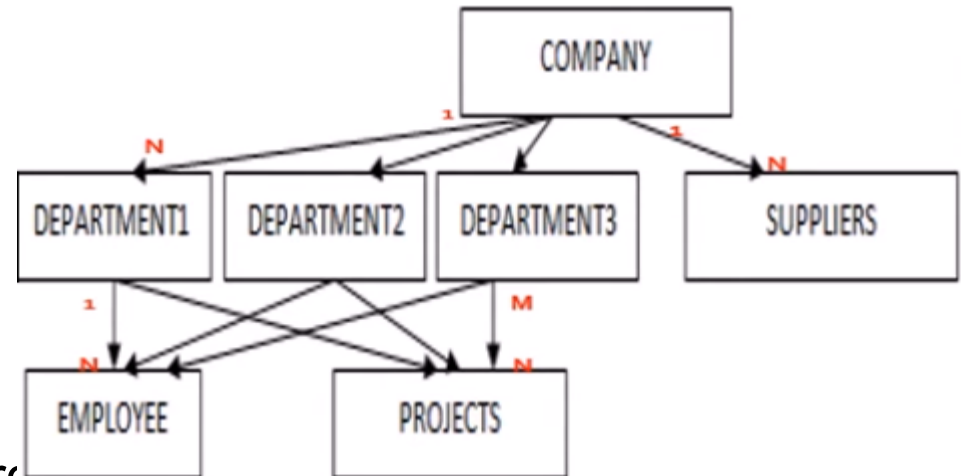
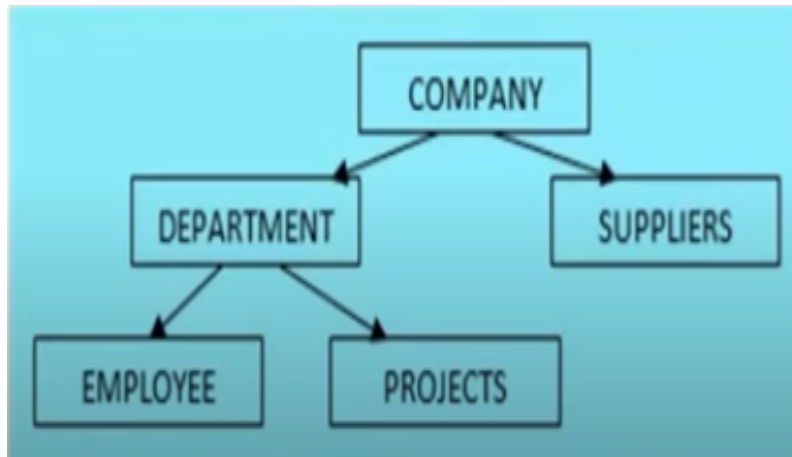
- Defines the actual relationship between the data in the entities
- Based on application and user levels of data
- Modeled considering the logical structure of the objects in the database
- Types
 - ✓ Hierarchical Data Models
 - ✓ Network Data Models
 - ✓ Relational Data Models

Record Based Data Models:



Hierarchical Data Model

- Data is organized into a tree-like structure
- Data is stored as records which are connected to one another through links
- A record is a collection of fields, with each field containing only one value



- Entities are represented in a hierarchical fashion
- Here a parent entity and its child entity are identified
- Again drill down is to be done to identify next level of child entity and soon

Hierarchical Data Model

Advantages

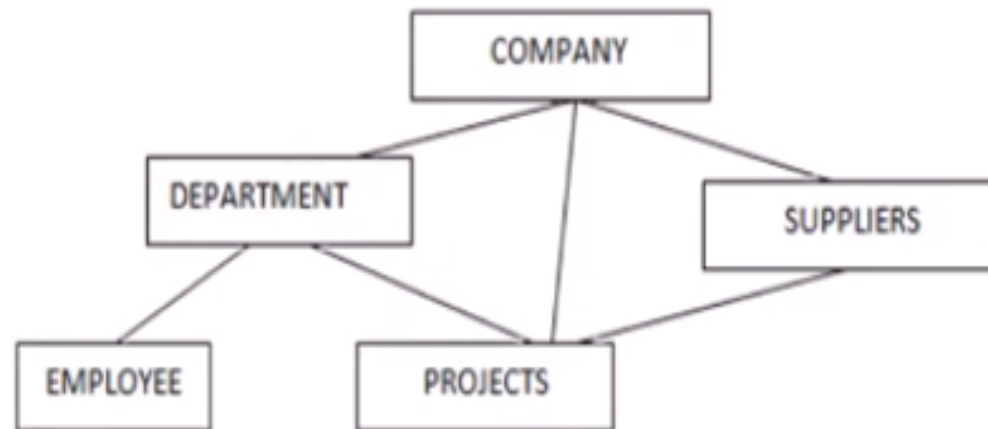
- Helps to address the issues of flat file data storage
- In flat files, data will be scattered and there will not be any proper structuring of the data
- Groups the related data into tables and defines the relationship between the tables, which is not addressed in flat files

Disadvantages

- Fails to handle many to many relationships efficiently. It results in redundancy and confusion
- Fetching through this model becomes bit difficult

Network Data Model

- Enhanced version of hierarchical data model
- Helps to address M:N relationship
- Also represented as hierarchical data model, but this model will not have single parent concept
- Any child in the tree can have multiple parents here



Network Data Model

Advantages

- Accessing the records in the tables is easy since it addresses many to many relationships
- One can easily navigate among the tables and get any data
- Designed based on database standards – ANSI/SPARC

Disadvantages

- No independence between any objects. Hence any changes to any of the object will need changes to the whole model
- Little difficult to design the relationship between the entities, since all the entities are related in someway

Relational Data Model

- Define how they are structured in the database physically and how they are inter-related
- Based on how the records in each table are related
- Isolates physical structure from the logical structure
- A relational data model revolves around 5 important rules
- ✓ Order of rows/records in the table is not important
- ✓ Each record in the table is unique
- ✓ Each column/attribute will have single value in a row
- ✓ All attributes should be from same domain
- ✓ Table names in the database should be unique

EMPLOYEE					DEPARTMENT		PROJECT	
EMP_ID	EMP_NAME	ADDRESS	DEPT_ID	PROJ_ID	DEPT_ID	DEPT_NAME	PROJ_ID	PROJ_NAME
100	Joseph	Clinton Town	10	206	10	Accounting	201	C Programming
101	Rose	Fraser Town	20	205	20	Quality	202	Web development
102	Mathew	Lakeside Village	10	206	30	Design	204	Database Design
103	Stewart	Troy	30	204			205	Testing
104	William	Holland	30	202			206	Pay Slip Generation

Relational Data Model

Advantages

- Structural Independence - Any changes to the database structure, does not affect the way of accessing the data
- Simplicity - Designed based on logical data and does not consider how data are stored physically in the memory
- ✓ Easy to maintain and access
- ✓ Supports Structured Query Language (SQL)

Disadvantages

- High hardware cost
- Lead to complexity in the database

ER Model to Relational Model Mapping

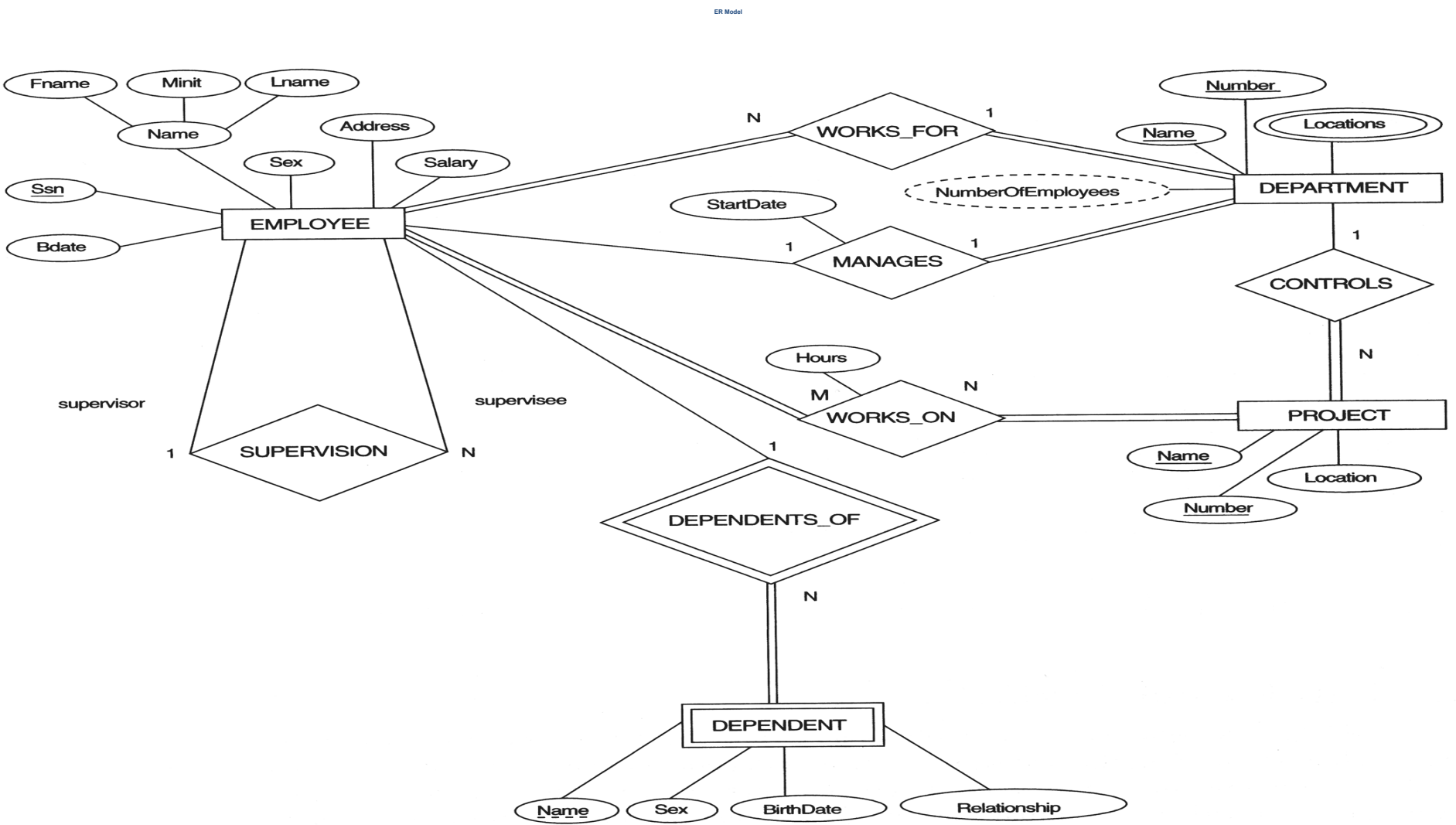
Steps in ER Model to Relational Model Mapping

- 7-Step Process
- ✓ Map Regular Entity Types
- ✓ Map Weak Entity Types
- ✓ Map Binary 1:1 Relation Types
- ✓ Map Binary 1:N Relationship Types
- ✓ Map Binary M:N Relationship Types
- ✓ Map Multivalued Attributes
- ✓ Map N-ary Relationship Types

ER Model to Relational Model Mapping:



ER Model to Relational Model Mapping



ER Model to Relational Model Mapping

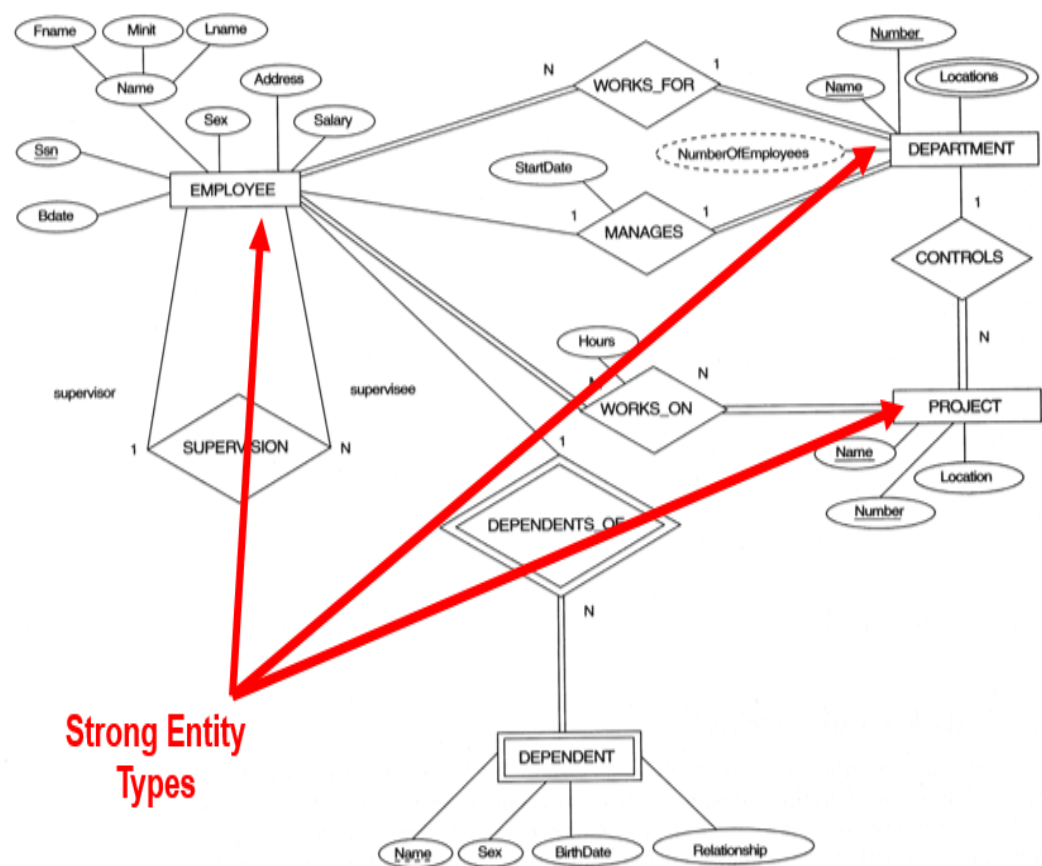
Step 1 - Mapping of Regular Entity Types

- For each regular (strong) entity type E in the ER schema, create a relation R that includes all the simple attributes of E
- Choose one of the key attributes of E as the primary key for R
- If the chosen key of E is composite, the set of simple attributes that form it will together form the primary key of R
- Relations EMPLOYEE, DEPARTMENT, and PROJECT in the relational schema corresponding to the regular entities in the ER diagram

ER Model to Relational Model Mapping

Step 1 - Mapping of Regular Entity Types

ER Model



Relational Model

EMPLOYEE							
Fname	Minit	Lname	Sex	Address	Salary	<u>Ssn</u>	Bdate
DEPARTMENT							
Name	<u>Number</u>						
PROJECT							
Name	<u>Number</u>	Location					

ER Model to Relational Model Mapping

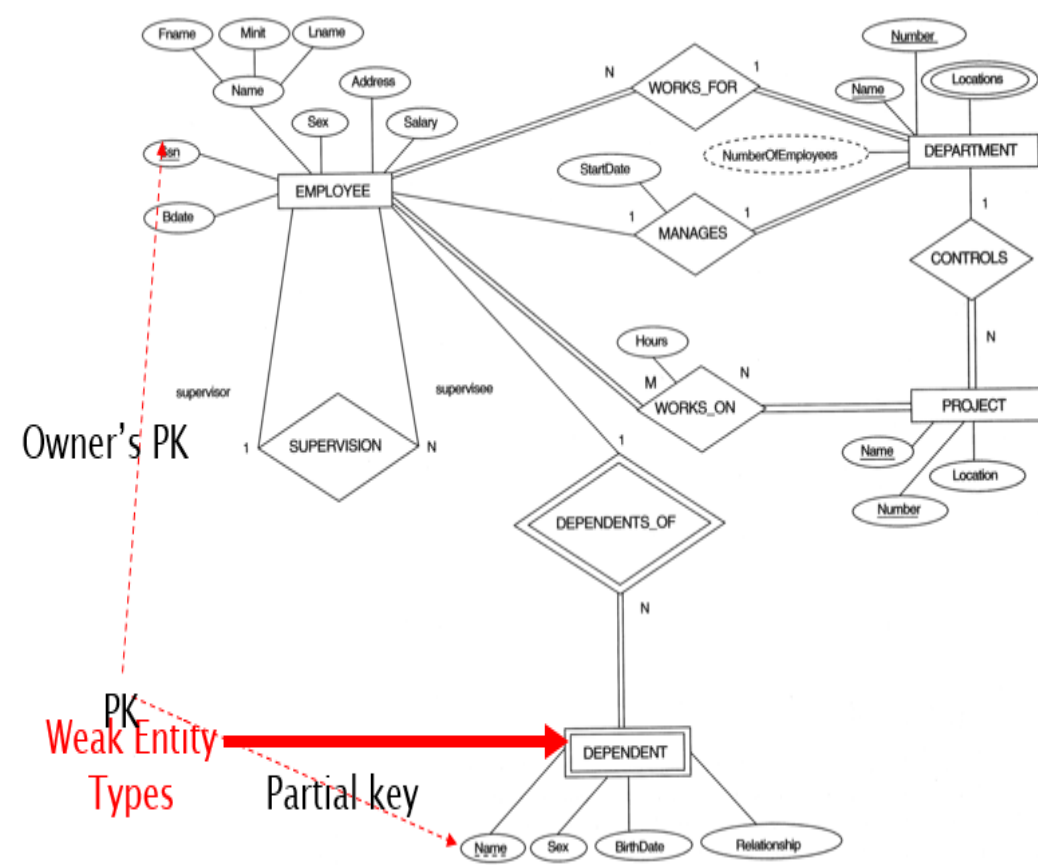
Step 2 - Mapping of Weak Entity Types

- For each weak entity type W in the ER schema with owner entity type E, create a relation R & include all simple attributes (or simple components of composite attributes) of W as attributes of R
- Also, include as foreign key attributes of R the primary key attribute(s) of the relation(s) that correspond to the owner entity type(s)
- The primary key of R is the combination of the primary key(s) of the owner(s) and the partial key of the weak entity type W, if any

ER Model to Relational Model Mapping

Step 2 - Mapping of Weak Entity Types

ER Model



Relational Model

EMPLOYEE							
Fname	Minit	Lname	Sex	Address	Salary	<u>Ssn</u>	Bdate

DEPARTMENT	
Name	<u>Number</u>

DEPENDENT				
<u>ESSN</u>	<u>Name</u>	Sex	Bdate	Relationship

PROJECT		
Name	<u>Number</u>	Location

ER Model to Relational Model Mapping

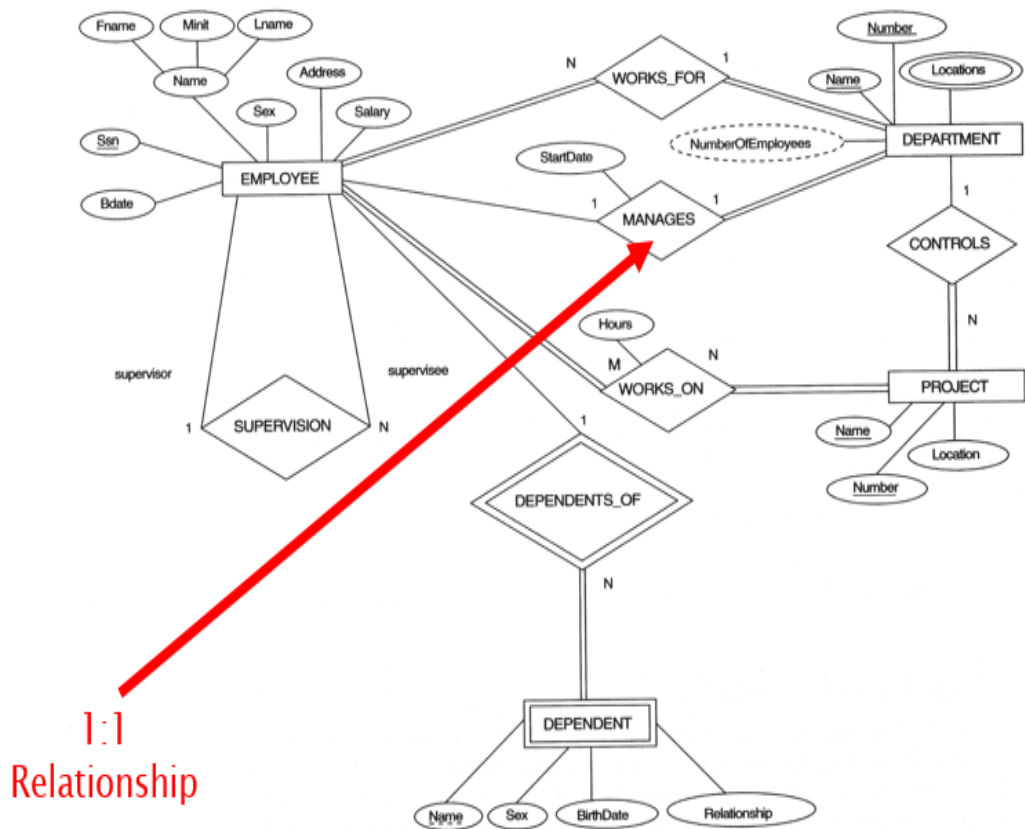
Step 3 - Mapping of Binary 1:1 Relation Types

- For 1:1 relationship key propagation may be used
- Relationship can be represented either putting the primary key of the first entity in the second entity or vice versa
- In some situations when mapping 1:1 relationship between entities, both entities and the relationship may be merged into one and represented by one relation

ER Model to Relational Model Mapping

Step 3 - Mapping of Binary 1:1 Relation Types

ER Model



Relational Model

EMPLOYEE							
Fname	Minit	Lname	Sex	Address	Salary	<u>Ssn</u>	Bdate
DEPARTMENT							
Name	<u>Number</u>	Mgr_Ssn	Mgr_StartDate				
DEPENDENT							
<u>ESSN</u>	<u>Name</u>	Sex	Bdate	Relationship			
PROJECT							
Name	<u>Number</u>	Location					

ER Model to Relational Model Mapping

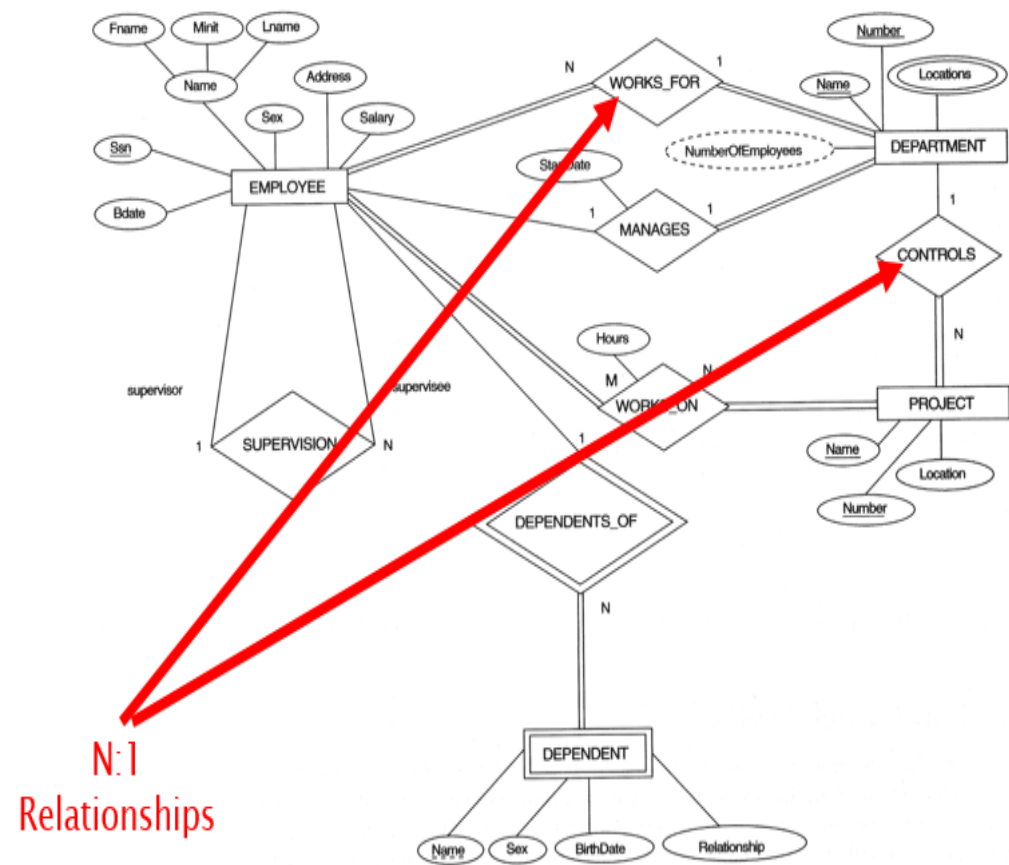
Step 4 - Mapping of Binary 1:N Relation Types

- One instance of the first entity may be related to a number of instances of the second entity
- But each instance of the second entity is related to only one instance of the first entity
- Key propagation is possible in 1:N relationship
- Propagation of primary key of first entity will happens.(E.g., From Department to Project)

ER Model to Relational Model Mapping

Step 4 - Mapping of Binary 1:N Relation Types

ER Model



Relational Model

EMPLOYEE								
Fname	Minit	Lname	Sex	Address	Salary	<u>Ssn</u>	Bdate	Dno

DEPARTMENT			
Name	<u>Number</u>	Mgr_Ssn	Mgr_StartDate

DEPENDENT				
<u>ESSN</u>	<u>Name</u>	Sex	Bdate	Relationship

PROJECT			
Name	<u>Number</u>	Location	Dno

Arrows indicate foreign key relationships: Dno in EMPLOYEE to Number in DEPARTMENT; Mgr_Ssn in DEPARTMENT to Ssn in EMPLOYEE; ESSN in DEPENDENT to Ssn in EMPLOYEE; Name in DEPENDENT to Name in PROJECT; and Number in PROJECT to Number in DEPARTMENT.

ER Model to Relational Model Mapping

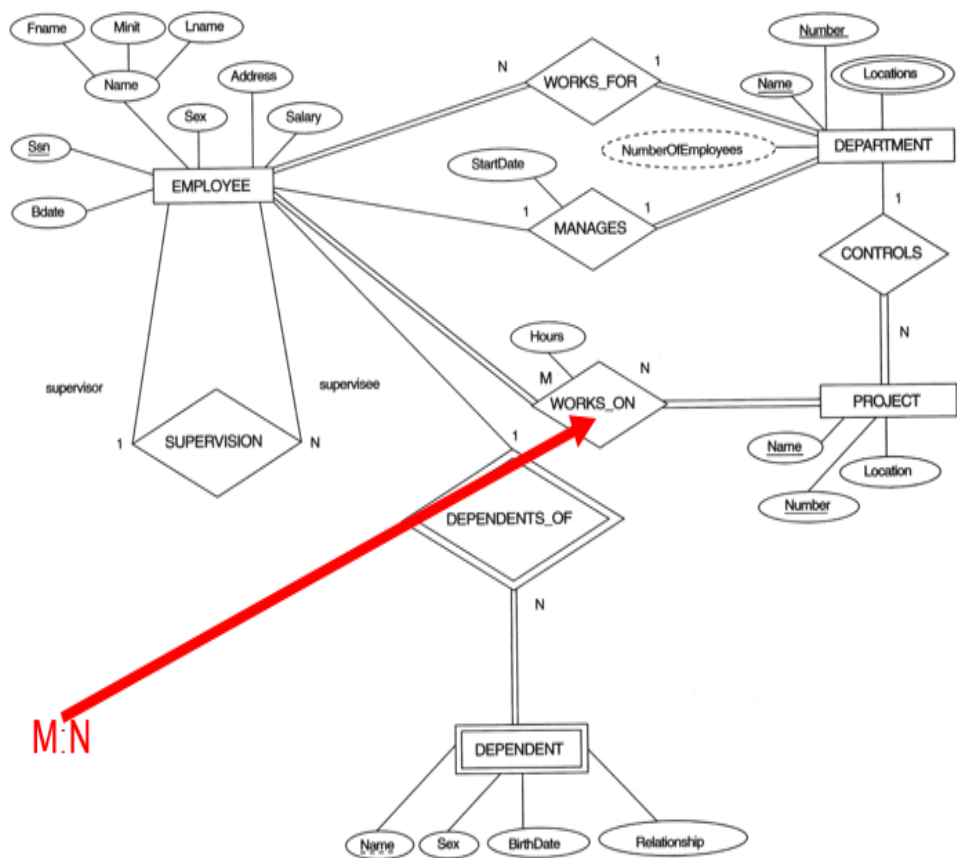
Step 5 - Mapping of Binary M:N Relation Types

- Key Propagation is not possible in many-to-many relationship
- **Many-to-Many relationships must be represented by a separate table**
- The primary keys of each of the entities that participates in the relationship are included in this relation
- The combination of the primary keys of entity will form the primary key of relation
- Any attributes of the relationship can be included in this relation

ER Model to Relational Model Mapping

Step 5 - Mapping of Binary M:N Relation Types

ER Model



Relational Model

EMPLOYEE								
Fname	Minit	Lname	Sex	Address	Salary	<u>Ssn</u>	Bdate	Dno
DEPARTMENT								
Name	<u>Number</u>	Mgr_Ssn	Mgr_StartDate					
DEPENDENT								
<u>ESSN</u>	<u>Name</u>	Sex	Bdate	Relationship				
PROJECT								
Name	<u>Number</u>	Location	Dno					
WORKS_ON								
Hours	<u>ESSN</u>	<u>Pno</u>						

ER Model to Relational Model Mapping

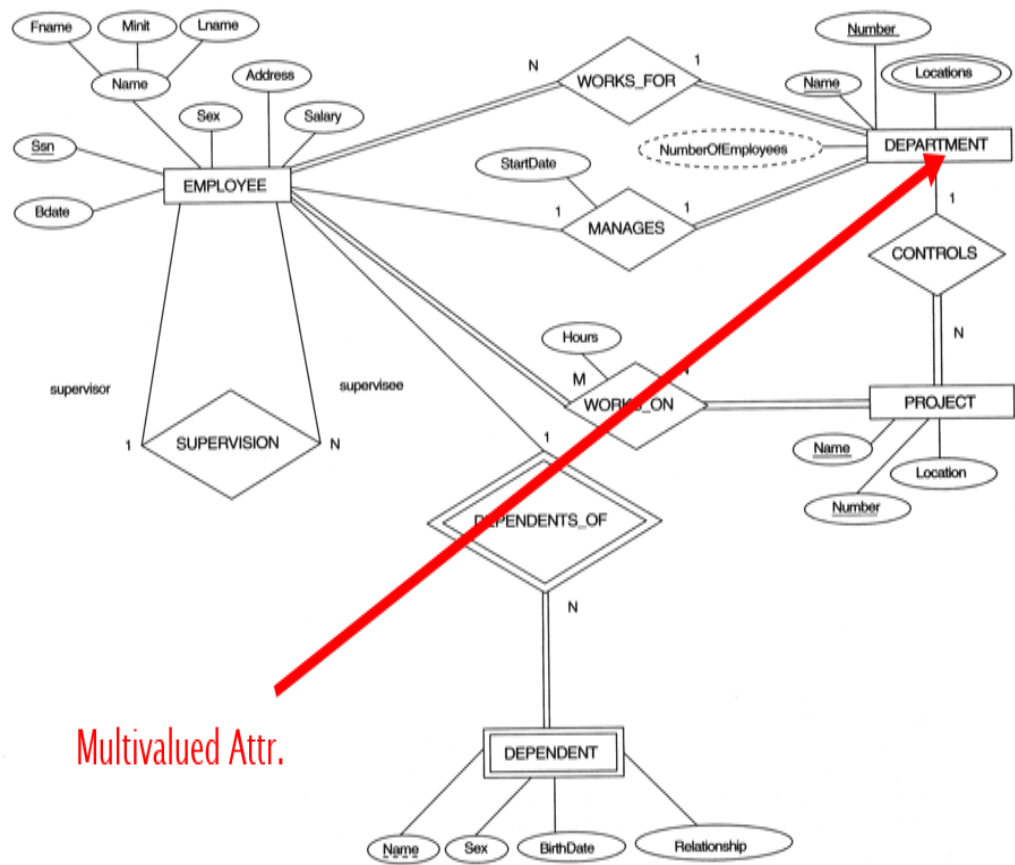
Step 6 - Mapping of Multivalued Attributes

- For each multivalued attribute A, create a new relation R
- This relation R will include an attribute corresponding to A, plus the primary key attribute K-as a foreign key in R-of the relation that represents the entity type of relationship type that has A as an attribute
- The primary key of R is the combination of A and K
- If the multivalued attribute is composite, include its simple components

ER Model to Relational Model Mapping

Step 6 - Mapping of Multivalued Attributes

ER Model



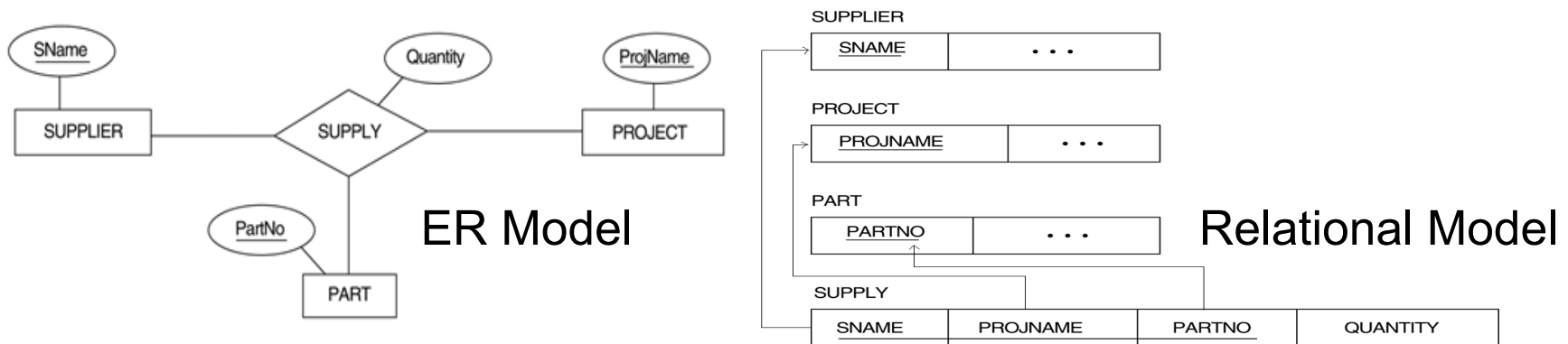
Relational Model

EMPLOYEE																											
Fname	Minit	Lname	Sex	Address	Salary	<u>Ssn</u>	Bdate	Dno	SuperSSN																		
DEPARTMENT																											
Name	<u>Number</u>	Mgr_Ssn	Mgr_StartDate																								
DEPENDENT																											
<u>ESSN</u>	<u>Name</u>	Sex	Bdate	Relationship																							
PROJECT																											
Name	<u>Number</u>	Location	Dno																								
WORKS_ON																											
Hours	<u>ESSN</u>	<u>Pno</u>																									
DEPT_LOCATION																											
<u>Dno</u>	<u>Location</u>																										

ER Model to Relational Model Mapping

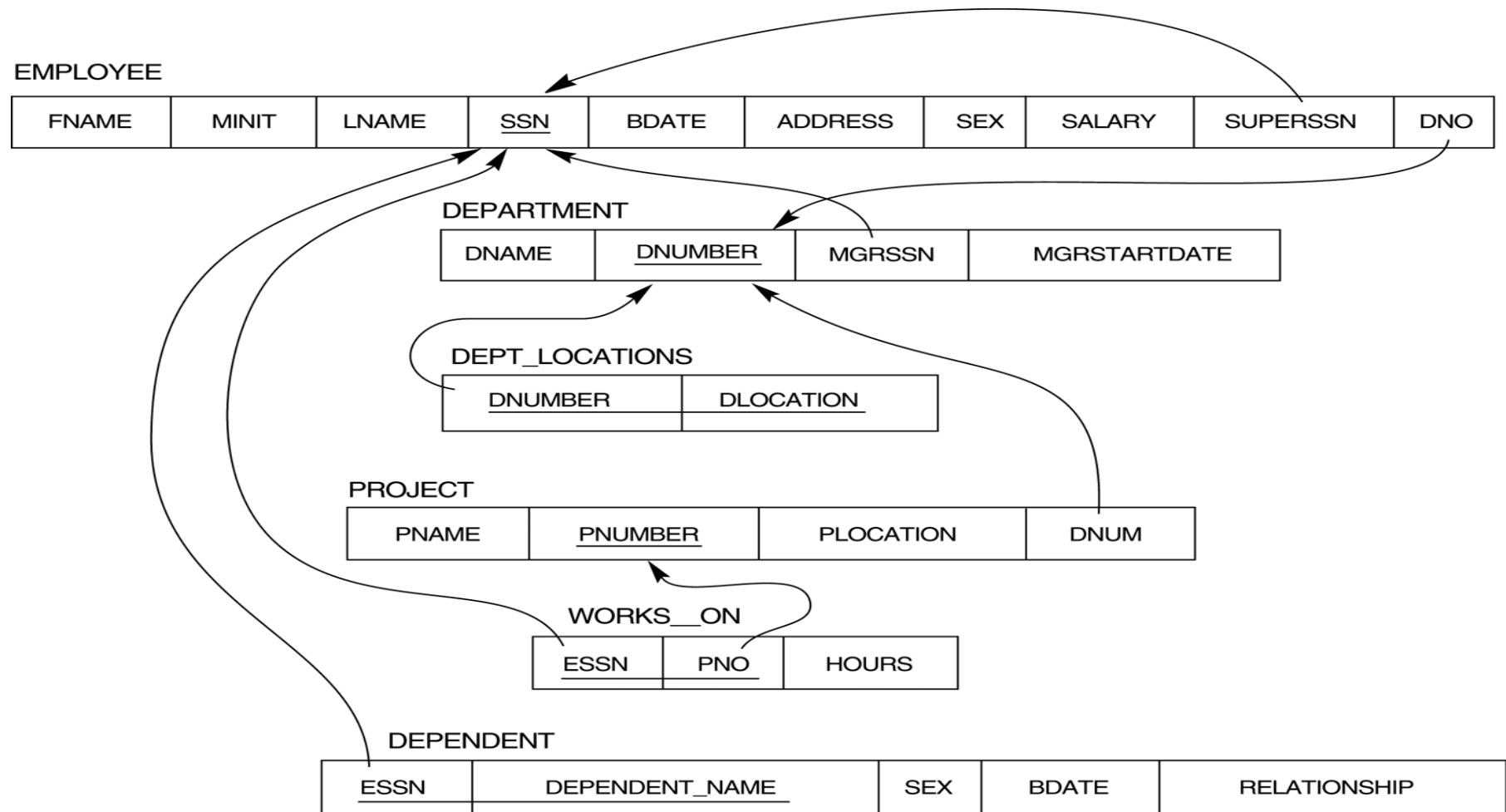
Step 7 - Mapping of N-ary Relationship Types

- For each n-ary relationship type R, where $n > 2$, create a new relationship S to represent R
- Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types
- Also include any simple attributes of the n-ary relationship type (or simple components of composite attributes) as attributes of S



ER Model to Relational Model Mapping

Relational Model



Introductions to Data Models-Hierarchical Model, Network Model, Object Oriented Model, Entity–Relationship Model, Relational Model

Summary

- Data Models
- Types of Data Models
- Physical Data Models
- Object Based Data Models
- Entity Relationship (ER) Data Model
- Object Oriented (OO) Data Model
- Record Based Data Models
- Hierarchical Data Model
- Network Data Model
- Relational Data Model
- ER Model to Relational Model Mapping

Introductions to Data Models-Hierarchical Model, Network Model, Object Oriented Model, Entity–Relationship Model, Relational Model

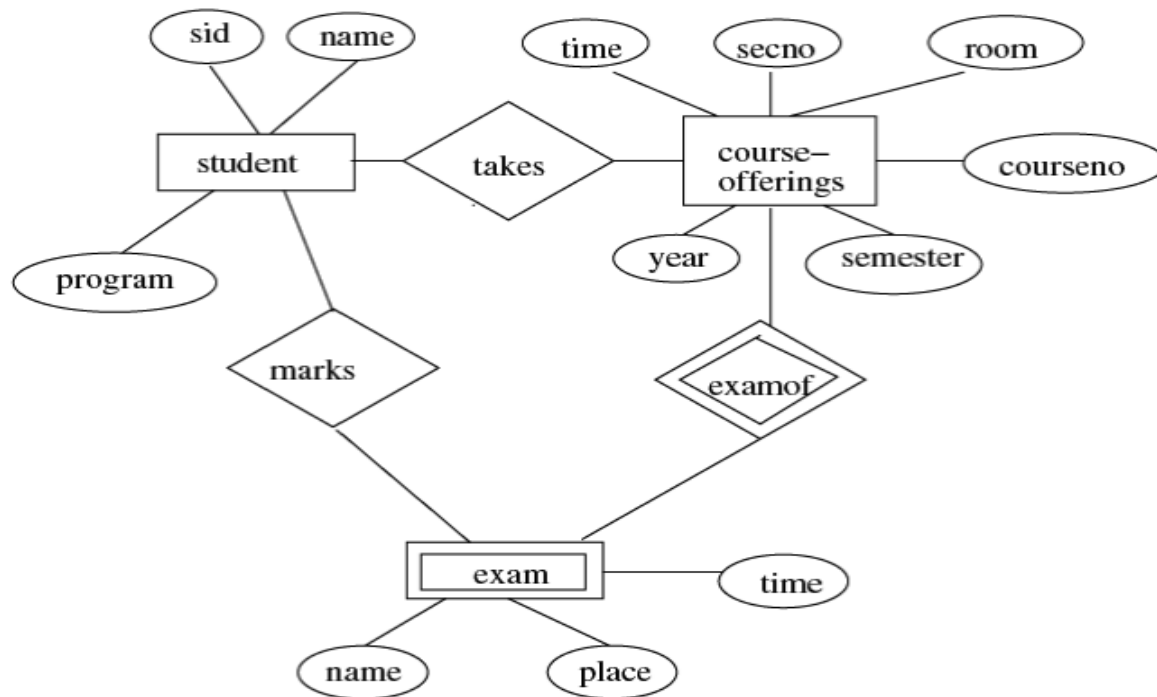
Try Yourself

- Draw an ER diagram for a University DataBase
- Draw a OO Data Model with the classes Person and Student
- Draw a Hierarchical Data Model for a Customer DataBase
- Draw a Network Data Model for a School DataBase
- Draw a Relational Data Model for a College DataBase

Introductions to Data Models-Hierarchical Model, Network Model, Object Oriented Model, Entity–Relationship Model, Relational Model

Try Yourself

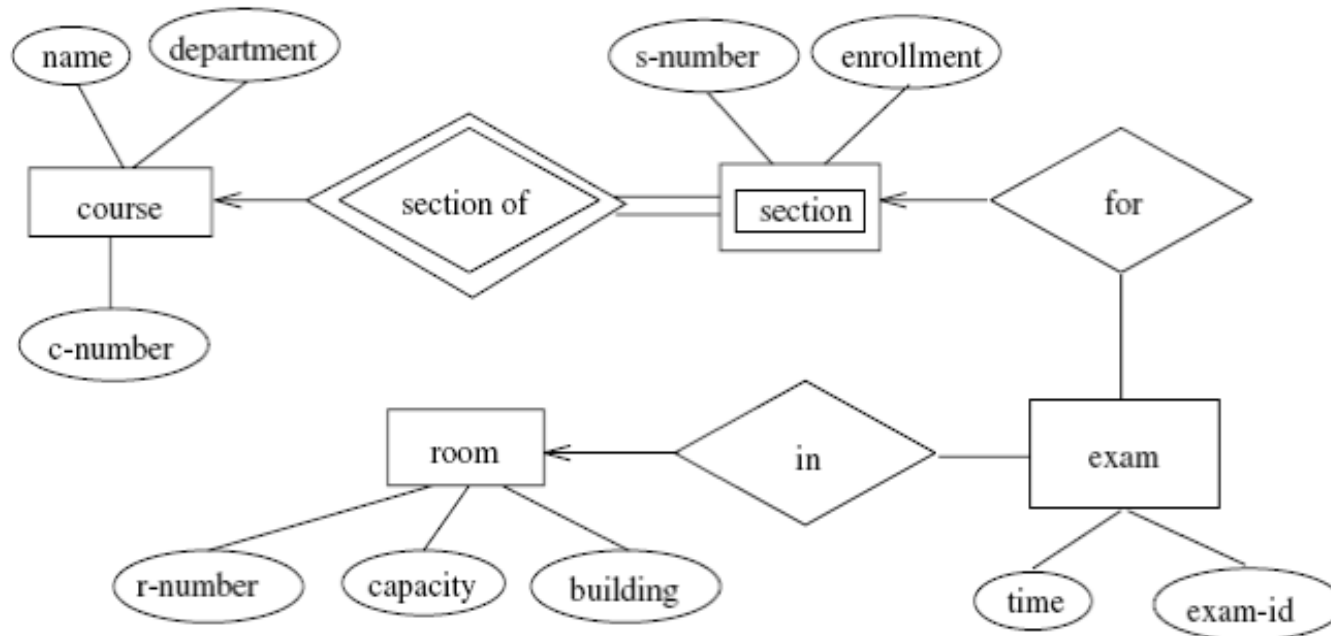
- Convert the following ER Model to Relational Schema:



Data Models – Introduction, Benefits, and Phases, ER Diagrams – Symbols, Components, Relationships, Weak entities, Attributes, Cardinality

Try Yourself

- Convert the following ER Model to Relational Schema:



Session 4 - Relational Algebra, Domain Relational Calculus, Tuple Relational Calculus